



Set Up Kubernetes Deployment



Ahmed Tetteh

```
[ec2-user@ip-172-31-34-127 nextwork-flask-backend]$ docker build -t nextwork-flask-backend .
[+] Building 1.0s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 269B
=> [internal] load metadata for docker.io/library/python:3.9-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 42.32kB
=> [1/5] FROM docker.io/library/python:3.9-alpine@sha256:c99b6eb43b3ac4d750db3d6e8b22268d5ea9a99deead7218ce3deda7f2ca029c
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY requirements.txt requirements.txt
=> CACHED [4/5] RUN pip3 install -r requirements.txt
=> CACHED [5/5] COPY . .
=> exporting to image
=> => exporting layers
=> => writing image sha256:4e7d0fef3d4efd4e5234eb87aa76a1961887cfec344602f73569f4449115198e
=> => naming to docker.io/library/nextwork-flask-backend
```



Introducing Today's Project!

In this project, I will clone a backend app from GitHub and deploy it with an EKS cluster.

Tools and concepts

I used Amazon EKS, Git, EC2 and ECR create to an EKS cluster for a backend application. Key steps include using "eksctl" manage the provisioning of the EKS cluster without writing too much code, and building and pushing the docker image to an ECR repository.

Project reflection

This project took me approximately 1hr 30 mins. The most challenging part was leaning how to find an easier way to provision the EKS cluster.

Something new that I learnt from this experience was utilizing the "eksctl" tool for cluster provisioning.

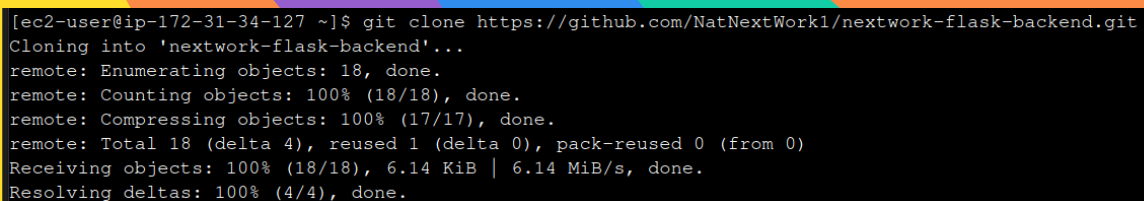


What I'm deploying

To set up today's project, I launched a Kubernetes cluster. Steps I took to do this included installing the "eksctl" tool, and using it to create an EKS cluster without using the AWS CLI. It simply detects all resources I need to provision and automates their creation with some few, short commands

I'm deploying an app's backend

Next, I retrieved the backend that I plan to deploy. An app's backend means the brain of the whole application - processing user requests, storing data and making sure everything is behaving as expected. I retrieved backend code by cloning it on the EC2 instance from GitHub



```
[ec2-user@ip-172-31-34-127 ~]$ git clone https://github.com/NatNextWork1/nextwork-flask-backend.git
Cloning into 'nextwork-flask-backend'...
remote: Enumerating objects: 18, done.
remote: Counting objects: 100% (18/18), done.
remote: Compressing objects: 100% (17/17), done.
remote: Total 18 (delta 4), reused 1 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (18/18), 6.14 KiB | 6.14 MiB/s, done.
Resolving deltas: 100% (4/4), done.
```



Building a container image

Once I cloned the backend code, my next step is to build a container image of the backend. This is because Kubernetes needs that image to deploy multiple instances of containers or pods on within the nodegroup.

When I tried to build a Docker image of the backend, I ran into a permissions error because Docker runs on the Kernel level, which requires root user permissions, something my default ec2-user does not have. Hence, I needed add the "sudo" command to elevate the user permissions and run the docker commands.

To solve the permissions error, I run the command "sudo usermod -aG docker ec2-user" to add the user to the docker group. The Docker group is is a group in Linux systems that gives users the permission to run Docker commands. This enables the ec2-user to have the permanent permissions it needs to run docker commands without using "sudo".



Ahmed Tetteh

NextWork Student

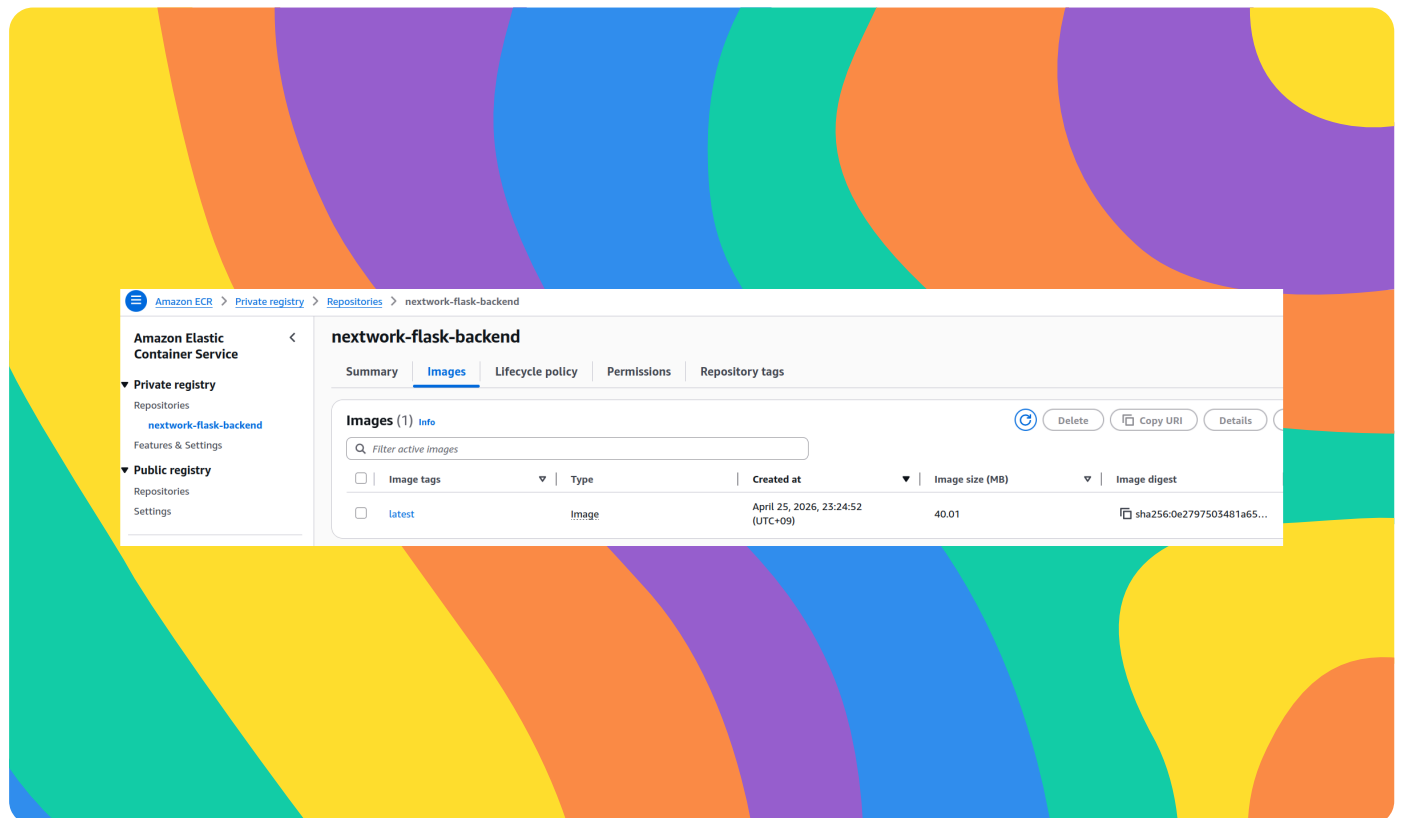
nextwork.org

```
[ec2-user@ip-172-31-34-127 nextwork-flask-backend]$ docker build -t nextwork-flask-backend .
[+] Building 1.0s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 269B
=> [internal] load metadata for docker.io/library/python:3.9-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 42.32kB
=> [1/5] FROM docker.io/library/python:3.9-alpine@sha256:c99b6eb43b3ac4d750db3d6e8b22268d5ea9a99deead7218ce3deda7f2ca029c
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY requirements.txt requirements.txt
=> CACHED [4/5] RUN pip3 install -r requirements.txt
=> CACHED [5/5] COPY . .
=> exporting to image
=> => exporting layers
=> => writing image sha256:4e7d0fef3d4efd4e5234eb87aa76a1961887cfec344602f73569f4449115198e
=> => naming to docker.io/library/nextwork-flask-backend
```

Container Registry

I'm using Amazon ECR in this project to store and deploy my image within the EKS cluster. ECR is a good choice for the job because it is an AWS service tightly integrated with EKS that makes versioning, sharing and deployment of images easier.

Container registries like Amazon ECR are great for Kubernetes deployment because they provide a fairly easy method of uploading multiple versions of tags of the same image, enabling consistency when pulling, and making sure your images are store securely.





EXTRA: Backend Explained

After reviewing the app's backend code, I've learnt that it creates an API that fetches data from the Hacker news API (external API) based on the user's search query (E.g. contents/ cloud), formats the response into JSON, and sends it back to the user or other services that require it.

Unpacking three key backend files

The requirements.txt file lists the app's dependencies.

The Dockerfile gives Docker instructions to docker on how to build the backend image. Key commands in this Dockerfile include FROM, WORKDIR, COPY, RUN and CMD.

The app.py file contains the main code for the backend. It loads up the libraries needed to create my own API that takes user input, connects to the Hacker News API to get the data, processes that data, and then sends it back as JSON.



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

